

Classes and object instances in Kotlin

About this codelab

1. Before you begin

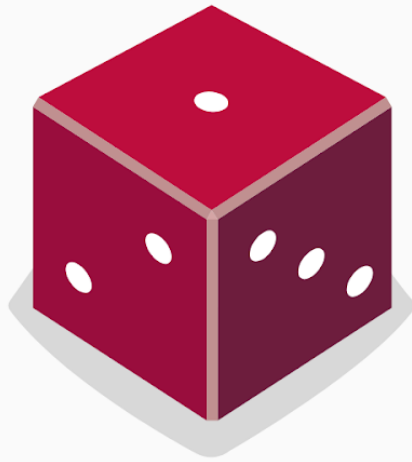
For the codelabs in this pathway, you will be building a Dice Roller Android app. When the user "rolls the dice," a random result will be generated. The result takes into account the number of sides of the dice. For example, only values from 1-6 can be rolled from a 6-sided dice.

This is what the final app will look like.

4:49



DiceRoller



ROLL



To help you focus on the new programming concepts for this app, you will use the browser-based Kotlin programming tool to create core app functionality. The program will output your results to the console. Later you will implement the user interface in Android Studio.

In this first codelab, you will create a Kotlin program that simulates rolling dice and outputs a random number, just like a dice would.

Prerequisites

- How to open, edit, and run code in <https://developer.android.com/training/kotlinplayground>
- Create and run a Kotlin program that uses variables and functions, and prints a result to the console.
- Format numbers within text using a string template with the `${variable}` notation.

What you'll learn

- How to programmatically generate random numbers to simulate dice rolls.
- How to structure your code by creating a `Dice` class with a variable and a method.
- How to create an object instance of a class, modify its variables, and call its methods.

What you'll build

- A Kotlin program in the browser-based Kotlin programming tool that can perform a random dice roll.

What you need

- A computer with an internet connection

2. Roll random numbers

Games often have a random element to them. You could earn a random prize or advance a random number of steps on the game board. In your everyday life, you can use random numbers and letters to generate safer passwords!

Instead of rolling actual dice, you can write a program that simulates rolling dice for you. Each time you roll the dice, the outcome can be any number within the range of possible values. Fortunately, you don't have to build your own random-number generator for such a program. Most programming languages, including Kotlin, have a built-in way for you to generate random numbers. In this task, you will use the Kotlin code to generate a random number.

Set up your starter code

1. In your browser, open the website <https://developer.android.com/training/kotlinplayground>.
2. Delete all the existing code in the code editor and replace it with the code below. This is the `main()` function you worked with in earlier codelabs.

```
fun main() {  
  
}
```

Use the random function

To roll a dice, you need a way to represent all the valid dice roll values. For a regular 6-sided dice, the acceptable dice rolls are: 1, 2, 3, 4, 5, and 6.

Previously, you learned that there are types of data like `Int` for integer numbers and `String` for text. `IntRange` is another data type, and it represents a range of integer numbers from a starting point to an endpoint. `IntRange` is a suitable data type for representing the possible values a dice roll can produce.

1. Inside your `main()` function, define a variable as a `val` called `diceRange`. Assign it to an `IntRange` from 1 to 6, representing the range of integer numbers that a 6-sided dice can roll.

```
val diceRange = 1..6
```

You can tell that `1..6` is a Kotlin range because it has a start number, two dots, followed by an ending number (no spaces in between). Other examples of integer ranges are `2..5` for the numbers 2 through 5, and `100..200` for the numbers 100 through 200.

Tip: Notice that it does not say `IntRange` in this definition, in the same way you did not have to specify `Int` or `String` when creating a variable for an integer or a string. Most of the time, the system can figure out what data type you intend.

For example, the system interprets this:

```
val diceRange = 1..6
```

as this:

```
val diceRange: IntRange = 1..6
```

Similar to how calling `println()` tells the system to print the given text, you can use a function called `random()` to generate and return a random number for you for a given range. As before, you can store the result in a variable.

2. Inside `main()`, define a variable as a `val` called `randomNumber`.

3. Make `randomNumber` have the value of the result of calling `random()` on the `diceRange` range, as shown below.

```
val randomNumber = diceRange.random()
```

Notice that you are calling `random()` on `diceRange` using a period, or dot, between the variable and the function call. You can read this as "generating a random number from `diceRange`". The result is then stored in the `randomNumber` variable.

4. To see your randomly generated number, use the string formatting notation (also called a "string template") `${randomNumber}` to print it, as shown below.

```
println("Random number: ${randomNumber}")
```

Your finished code should look like this.

```
fun main() {  
    val diceRange = 1..6  
    val randomNumber = diceRange.random()  
    println("Random number: ${randomNumber}")  
}
```

5. Run your code several times. Each time, you should see output as below, with different random numbers.

```
Random number: 4
```

Tips about ranges:

- Ranges can be between any integers. The following are valid ranges: `3..46`, `0..270`, `-6..+6`, `-10..-4`.
- You can call functions directly on a range, for example: `(1..6).random()`.

3. Create a Dice class

When you roll dice, they are real objects in your hands. While the code you just wrote works perfectly fine, it's hard to imagine that it's about actual dice. Organizing a program to be more like the things it represents makes it easier to understand. So, it would be cool to have programmatic dice that you can roll!

All dice work essentially the same. They have the same properties, such as sides, and they have the same behavior, such as that they can be rolled. In Kotlin, you can create a programmatic blueprint of a dice that says that dice have sides and can roll a random number. This blueprint is called a *class*.

From that class, you can then create actual dice objects, called *object instances*. For example, you can create a 12-sided dice, or a 4-sided dice.

Tip: A *class* is similar to how an architect's blueprint plans are not the house; they are the instructions of how to build the house. The house is the actual thing, or *object instance*, created according to the blueprint.

Note: Organizing everything about dice into a class is called encapsulation. *Encapsulation* is a big fancy word, but all it means is that you can enclose functionality that is logically related into a single place.

Define a Dice class

In the following steps, you will define a new class called `Dice` to represent a rollable dice.

1. To start afresh, clear out the code in the `main()` function so that you end up with the code as shown below.

```
fun main() {  
  
}
```

2. Below this `main()` function, add a blank line, and then add code to create the `Dice` class. As shown below, start with the keyword `class`, followed by the name of the class, followed by an opening and closing curly brace. Leave space in between the curly braces to put your code for the class.

```
class Dice {  
  
}
```

Note:

- Similar to using the `fun` keyword in Kotlin to create a new function, use the `class` keyword to create a new class.
- You can choose any name for a `class`, but it is helpful if the name indicates what the class represents. By convention, the class name is written in Upper Camel Case (also called Pascal Casing). For example: `Car`, `ParkingMeter`, and `CustomerRecord` are all valid class names, and you can guess at what they represent.

Inside a class definition, you can specify one or more properties for the class using variables. Real dice can have a number of sides, a color, or a weight. In this task, you'll focus on the property of number of sides of the dice.

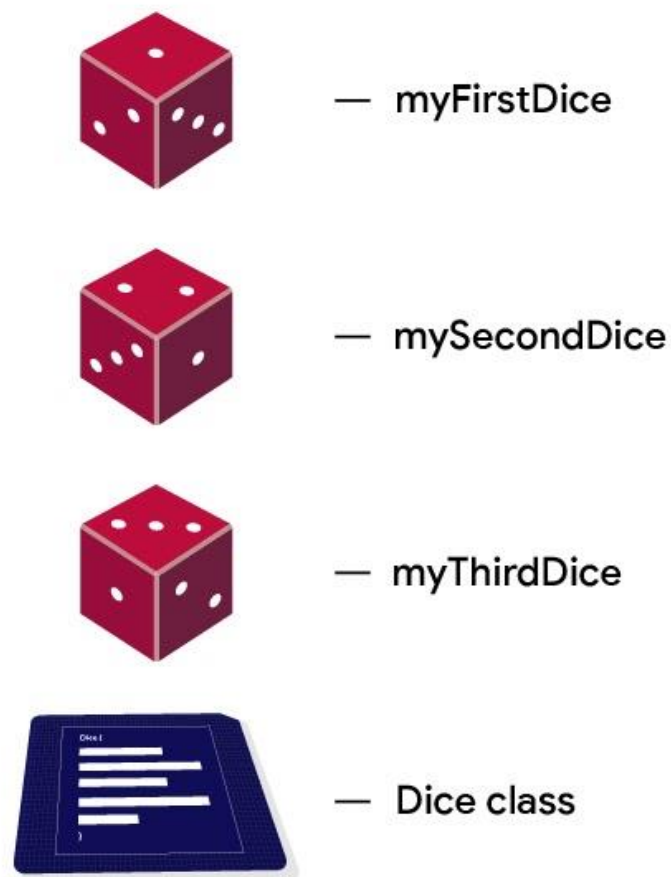
3. Inside the `Dice` class, add a `var` called `sides` for the number of sides your dice will have. Set `sides` to 6.

```
class Dice {  
    var sides = 6  
}
```

That's it. You now have a very simple class representing dice.

Create an instance of the Dice class

With this `Dice` class, you have a blueprint of what a dice is. To have an actual dice in your program, you need to create a `Dice` object instance. (And if you needed to have three dice, you would create three object instances.)



1. To create an object instance of `Dice`, in the `main()` function, create a `val` called `myFirstDice` and initialize it as an instance of the `Dice` class. Notice the parentheses

after the class name, which denote that you are creating a new object instance from the class.

```
fun main() {  
    val myFirstDice = Dice()  
}
```

Now that you have a `myFirstDice` object, a thing made from the blueprint, you can access its properties. The only property of `Dice` is its `sides`. You access a property using the "dot notation". So, to access the `sides` property of `myFirstDice`, you call `myFirstDice.sides` which is pronounced "`myFirstDice` dot `sides`".

2. Below the declaration of `myFirstDice`, add a `println()` statement to output the number of `sides` of `myFirstDice`.

```
println(myFirstDice.sides)
```

Note: You used the dot notation earlier when calling `diceRange.random()`. Generalized, you can think of the dot notation as saying, "on something-dot-perform some action". Such as here, with `myFirstDice.sides`, "on this instance, get the `sides` property".

Your code should look like this.

```
fun main() {  
    val myFirstDice = Dice()  
    println(myFirstDice.sides)  
}  
  
class Dice {  
    var sides = 6  
}
```

3. Run your program and it should output the number of `sides` defined in the `Dice` class.

6

You now have a `Dice` class and an actual dice `myFirstDice` with 6 sides.

Let's make the dice roll!

Make the Dice Roll

You previously used a function to perform the action of printing cake layers. Rolling dice is also an action that can be implemented as a function. And since all dice can be rolled, you can add a function for it inside the `Dice` class. A function that is defined inside a class is also called a *method*.

1. In the `Dice` class, below the `sides` variable, insert a blank line and then create a new function for rolling the dice. Start with the Kotlin keyword `fun`, followed by the name of the method, followed by parentheses `()`, followed by opening and closing curly braces `{}`. You can leave a blank line in between the curly braces to make room for more code, as shown below. Your class should look like this.

```
class Dice {  
    var sides = 6  
  
    fun roll() {  
  
    }  
}
```

Note: You can name this method anything you want, but it is helpful to give it a name that indicates what action it performs. The naming convention for functions and methods is to start with a lowercase letter, use camel case, and start with an action verb, if possible.

When you roll a six-sided dice, it produces a random number between 1 and 6.

2. Inside the `roll()` method, create a `val randomNumber`. Assign it a random number in the `1..6` range. Use the dot notation to call `random()` on the range.

```
val randomNumber = (1..6).random()
```

3. After generating the random number, print it to the console. Your finished `roll()` method should look like the code below.

```
fun roll() {  
    val randomNumber = (1..6).random()  
    println(randomNumber)  
}
```

4. To actually roll `myFirstDice`, in `main()`, call the `roll()` method on `myFirstDice`. You call a method using the "dot notation". So, to call the `roll()` method of `myFirstDice`, you type `myFirstDice.roll()` which is pronounced "myFirstDice dot roll()".

```
myFirstDice.roll()
```

Your completed code should look like this.

```
fun main() {  
    val myFirstDice = Dice()  
    println(myFirstDice.sides)  
    myFirstDice.roll()  
}  
  
class Dice {  
    var sides = 6
```

```

fun roll() {
    val randomNumber = (1..6).random()
    println(randomNumber)
}

```

5. Run your code! You should see the result of a random dice roll below the number of sides. Run your code several times, and notice that the number of sides stays the same, and the dice roll value changes.

```

6
4

```

Congratulations! You have defined a `Dice` class with a `sides` variable and a `roll()` function. In the `main()` function, you created a new `Dice` object instance and then you called the `roll()` method on it to produce a random number.

4. Return your dice roll's value

Currently you are printing out the value of the `randomNumber` in your `roll()` function and that works great! But sometimes it's more useful to return the result of a function to whatever called the function. For example, you could assign the result of the `roll()` method to a variable, and then move a player by that amount! Let's see how that's done.

1. In `main()` modify the line that says `myFirstDice.roll()`. Create a `val` called `diceRoll`. Set it equal to the value returned by the `roll()` method.

```
val diceRoll = myFirstDice.roll()
```

This doesn't do anything yet, because `roll()` doesn't return anything yet. In order for this code to work as intended, `roll()` has to return something.

In previous codelabs you learned that you need to specify a data type for input arguments to functions. In the same way, you have to specify a data type for data that a function returns.

2. Change the `roll()` function to specify what type of data will be returned. In this case, the random number is an `Int`, so the return type is `Int`. The syntax for specifying the return type is: After the name of the function, after the parentheses, add a colon, space, and then the `Int` keyword for the return type of the function. The function definition should look like the code below.

```
fun roll(): Int {
```

3. Run this code. You will see an error in the **Problems View**. It says:

```
A 'return' expression required in a function with a block body ('{...}')
```

You changed the function definition to return an `Int`, but the system is complaining that your code doesn't actually return an `Int`. "Block body" or "function body" refers to the code between the curly braces of a function. You can fix this error by returning a value from a function using a `return` statement at the end of the function body.

4. In `roll()`, remove the `println()` statement and replace it with a `return` statement for `randomNumber`. Your `roll()` function should look like the code below.

```
fun roll(): Int {  
    val randomNumber = (1..6).random()  
    return randomNumber  
}
```

5. In `main()` remove the print statement for the sides of the dice.
6. Add a statement to print out the value of `sides` and `diceRoll` in an informative sentence. Your finished `main()` function should look similar to the code below.

```
fun main() {  
    val myFirstDice = Dice()  
    val diceRoll = myFirstDice.roll()  
    println("Your ${myFirstDice.sides} sided dice rolled ${diceRoll}!")  
}
```

7. Run your code and your output should be like this.

Your 6 sided dice rolled 4!

Here is all your code so far.

```
fun main() {  
    val myFirstDice = Dice()  
    val diceRoll = myFirstDice.roll()  
    println("Your ${myFirstDice.sides} sided dice rolled ${diceRoll}!")  
}  
  
class Dice {  
    var sides = 6  
  
    fun roll(): Int {  
        val randomNumber = (1..6).random()  
        return randomNumber  
    }  
}
```

[5. Change the number of sides on your dice](#)

Not all dice have 6 sides! Dice come in all shapes and sizes: 4 sides, 8 sides, up to 120 sides!

1. In your `Dice` class, in your `roll()` method, change the hard-coded `1..6` to use `sides` instead, so that the range, and thus the random number rolled, will always be right for the number of sides.

```
val randomNumber = (1..sides).random()
```

2. In the `main()` function, below and after printing the dice roll, change `sides` of `myFirstDice` to be set to 20.

```
myFirstDice.sides = 20
```

3. Copy and paste the existing print statement below after where you changed the number of sides.
4. Replace the printing of `diceRoll` with printing the result of calling the `roll()` method on `myFirstDice`.

```
println("Your ${myFirstDice.sides} sided dice rolled ${myFirstDice.roll()}!")
```

Your program should look like this.

```
fun main() {  
  
    val myFirstDice = Dice()  
    val diceRoll = myFirstDice.roll()  
    println("Your ${myFirstDice.sides} sided dice rolled ${diceRoll}!")  
  
    myFirstDice.sides = 20  
    println("Your ${myFirstDice.sides} sided dice rolled  
${myFirstDice.roll()}!")  
}  
  
class Dice {  
    var sides = 6  
  
    fun roll(): Int {  
        val randomNumber = (1..sides).random()  
        return randomNumber  
    }  
}
```

5. Run your program and you should see a message for the 6-sided dice, and a second message for the 20-sided dice.

```
Your 6 sided dice rolled 3!  
Your 20 sided dice rolled 15!
```

6. Customize your dice

The idea of a class is to represent a thing, often something physical in the real world. In this case, a `Dice` class does represent a physical dice. In the real world, dice cannot change their number of

sides. If you want a different number of sides, you need to get a different dice. Programmatically, this means that instead of changing the `sides` property of an existing `Dice` object instance, you should create a new dice object instance with the number of sides you need.

In this task, you are going to modify the `Dice` class so that you can specify the number of sides when you create a new instance. Change the `Dice` class definition so you can supply the number of sides. This is similar to how a function can accept arguments for input.

1. Modify the `Dice` class definition to accept an integer called `numSides`. The code inside your class does not change.

```
class Dice(val numSides: Int) {  
    // Code inside does not change.  
}
```

2. Inside the `Dice` class, delete the `sides` variable, as you can now use `numSides`.
3. Also, fix the range to use `numSides`.

Your `Dice` class should look like this.

```
class Dice (val numSides: Int) {  
  
    fun roll(): Int {  
        val randomNumber = (1..numSides).random()  
        return randomNumber  
    }  
}
```

If you run this code, you will see a lot of errors, because you need to update `main()` to work with the changes to the `Dice` class.

4. In `main()`, to create `myFirstDice` with 6 sides, you must now supply in the number of sides as an argument to the `Dice` class, as shown below.

```
val myFirstDice = Dice(6)
```

5. In the print statement, change `sides` to `numSides`.
6. Below that, delete the code that changes `sides` to 20, because that variable does not exist anymore.
7. Delete the `println` statement underneath it as well.

Your `main()` function should look like the code below, and if you run it, there should be no errors.

```
fun main() {  
    val myFirstDice = Dice(6)  
    val diceRoll = myFirstDice.roll()  
    println("Your ${myFirstDice.numSides} sided dice rolled ${diceRoll}!")  
}
```

8. After printing the first dice roll, add code to create and print a second `Dice` object called `mySecondDice` with 20 sides.

```
val mySecondDice = Dice(20)
```

9. Add a print statement that rolls and prints the returned value.

```
println("Your ${mySecondDice.numSides} sided dice  
rolled ${mySecondDice.roll()}!")
```

10. Your finished `main()` function should look like this.

```
fun main() {  
    val myFirstDice = Dice(6)  
    val diceRoll = myFirstDice.roll()  
    println("Your ${myFirstDice.numSides} sided dice rolled ${diceRoll}!")  
  
    val mySecondDice = Dice(20)  
    println("Your ${mySecondDice.numSides} sided dice rolled  
${mySecondDice.roll()}!")  
}  
  
class Dice (val numSides: Int) {  
  
    fun roll(): Int {  
        val randomNumber = (1..numSides).random()  
        return randomNumber  
    }  
}
```

11. Run your finished program, and your output should look like this.

```
Your 6 sided dice rolled 5!  
Your 20 sided dice rolled 7!
```

7. Adopt good coding practices

When writing code, concise is better. You can get rid of the `randomNumber` variable and return the random number directly.

1. Change the `return` statement to return the random number directly.

```
fun roll(): Int {  
    return (1..numSides).random()  
}
```

In the second print statement, you put the call to get the random number into the string template. You can get rid of the `diceRoll` variable by doing the same thing in the first print statement.

2. Call `myFirstDice.roll()` in the string template and delete the `diceRoll` variable. The first two lines of your `main()` code now look like this.

```
val myFirstDice = Dice(6)
println("Your ${myFirstDice.numSides} sided dice rolled
${myFirstDice.roll()}!")
```

3. Run your code and there should be no difference in the output.

Note: Changing code to make it shorter, more efficient, or easier to read and understand is called *refactoring*. It's like writing a document, where you write a first draft that has all the information, and then edit and refine your words.

This is your final code after *refactoring* it .

```
fun main() {
    val myFirstDice = Dice(6)
    println("Your ${myFirstDice.numSides} sided dice rolled
${myFirstDice.roll()}!")

    val mySecondDice = Dice(20)
    println("Your ${mySecondDice.numSides} sided dice rolled
${mySecondDice.roll()}!")
}

class Dice (val numSides: Int) {

    fun roll(): Int {
        return (1..numSides).random()
    }
}
```

8. Solution code

```
fun main() {
    val myFirstDice = Dice(6)
    println("Your ${myFirstDice.numSides} sided dice rolled
${myFirstDice.roll()}!")

    val mySecondDice = Dice(20)
    println("Your ${mySecondDice.numSides} sided dice rolled
${mySecondDice.roll()}!")
}

class Dice (val numSides: Int) {

    fun roll(): Int {
        return (1..numSides).random()
    }
}
```

9. Summary

- Call the `random()` function on an `IntRange` to generate a random number:
`(1..6).random()`
- Classes are like a blueprint of an object. They can have properties and behaviors, implemented as variables and functions.
- An instance of a class represents an object, often a physical object, such as a dice. You can call the actions on the object and change its attributes.
- You can supply values to a class when you create an instance. For example: `class Dice(val numSides: Int)` and then create an instance with `Dice(6)`.
- Functions can return something. Specify the data type to be returned in the function definition, and use a `return` statement in the function body to return something. For example: `fun example(): Int { return 5 }`

10. Learn more

- [Vocabulary for Android Basics in Kotlin](#)
- [Random number generation \(Wikipedia\)](#)
- [The Mind-Boggling Challenge of Designing 120-sided Dice](#)
- [Classes in Kotlin](#)
- [Function declarations in Kotlin](#)
- [Returning a value from a function](#)

11. Practice on your own

Note: Practices are optional. They provide an opportunity for you to practice what you've learned in this codelab.

Do the following:

- Give your `Dice` class another attribute of color and create multiple instances of dice with different numbers of sides and colors!
- Create a `Coin` class, give it the ability to flip, create an instance of the class and flip some coins! How would you use the `random()` function with a range to accomplish the coin flip?